```
public class Demo

{
public staic void main(string[]args){
  byte b =127; //125;
  int a = b;

    system.out.println(a);
    }
    }



    int a = 12;
    byte k = (byte)a;

    system.out.println(k);
    }
    }


    int a = 257;
    byte k = (byte) a;
     float f  = 5.6f;
     int t =(int) f;
     system.out.println(t);
     }
     }


     byte = 10;
     byte = 30;
     int result = a*b;
     system.out.println(result);
     }
     }

9.   Arithmetic Operator


     1. Assignment Operator
     public class Arithmetic
     {
     public static void main(String[] args){
      int num1 =7;
      int num2 = 5;

      int result = num1 + num2;
      system.out.println(result);
      }
      }

     int result = num1-num2;


     int result = num1*num2;

     int result = num1/num2;

     int result = num1%num2


     // int num1  = 7;
     // num1 = num2+2;

     // num1 +=2;
     // num1 +=1;
```

```
      // num1++;  // post- Increment
      ++num1;     // pre- Increment
      // num1--;     // Decrement
       t  result = num++; // fetch the value and then increment f
      system.out.println(num1);
      }
      }



    * Relational Operator

    pulic class Relational
    {
    public static void main(String[] args){

    int x = 6; // double x = 8.8;
    int y = 5;  // double y = 9.8;

//  boolean result = x < y;

//  boolean result = x !=y;
boolean result = x == y;


      system.out.println(result);
      }
      }


       *LOGICAL OPERATOR IN JAVA

         combine two different logical operator in between

         x < y  And/OR   a >b

         T/F             T/F

         &- And      |- OR          !- Not

         T T - T     T T -T        T - F

         T F - F     T F - T       F - T
         F T - F     F T - T
         F F - F     F F - F

         &  |  !
         && || !

         r = x > y && a < b
         T      T       T

         x = x > y || a < b
         T     T        T

          r = x > y && a < b
          T     F        T

          example:-

          public class Demo
          {
          public staic void main (String [] args){

            int x = 7;
            int y = 5;
            int a = 5;
```

```
            int b = 9;

         //  boolean result = x > y ;
      //  boolean result = x > y && a > b;
  //  boolean result = x > y || a > b;
        boolean result = x > y || a < b || a > 1;
        boolean result = a > b;

              System.out.println(result);
              }
              }


              * IF Else in java

              public class ifElse
              {
              public static void main(String[]args){

              int x = 8;
              int y = 7; // 17
              int z = 5; // int z = 9;

              if(x>y && x>z)
              System.out.println(x);
              else  if(y>x && y>z)
                System.out.println(y);
                else
                  System.out.println(z);
                }
              }


              14. Ternary Operator in java

              public class ifElse
              {
              public static void main(String[]args){

                   int n = 4; // 5;
                   int result = 0;

                   if (n%2==0)
                   result = 10;
                   else
                   result = 20;

                   system.out.println(result);
                   }

                      }



      2nd Method

       public class ifElse
              {
              public static void main(String[]args){

              int n = 4;
              int result = 0;


                // ?:

                result = n%2==0 ? 10 : 20;
```

```
            system.out.println(result);
            }
            }
```

15. Switch Statement in java

Switch Statement :- switch is multiple choice decision making selection statement, it used when we want to select only one case out of muliple cases.

```
 public class SwitchStatement
 {
 public Static void main(String[]args){

    int n = 1;

    System.out.println("Monday");
    System.out.println("Tuesday");
    System.out.println("Wednesday");
    System.out.println("Thursday");
    System.out.println("Friday");
    System.out.println("Saturday");
    System.out.println("Sunday");

 }
 }

 //  public class SwitchStatement
 {
 public Static void main(String[]args){

    int n = 2; // int n = 8;

    switch (n){

case 1:
    System.out.println("Monday");
     break;
case 2:
    System.out.println("Tuesday");
     break;
case 3:
    System.out.println("Wednesday");
     break;
case 4:
    System.out.println("Thursday");
     break;
case 5:
    System.out.println("Friday");
     break;
case 6:
    System.out.println("Saturday");
     break;
case 7:
    System.out.println("Sunday");
    break;
    default:
     System.out.println("Enter a default number");

 }
 }
```

#. What's new in java switch Statement and Expression

```
 public class switch
```

```
          {

          public static void main(String []agrs){

            string day = "Monday";

            switch(day)
            {
             case "Satuday", "Sunday":
             System.out.println("6am");
             break;
             case "Monday":
             System.out.println("8am");

             default:
             System.out.println("7am");
             }
             }


    16. Need For Loop in java

        For Loop:- for loop is the most commonly used loop.  it is used when we want to
        perform initialization,condition and incr|decr operation in single line


    package Loop;

public class Loop {
    public static void main(String[]args) {
        System.out.println("hi");
        System.out.println("hi");
        System.out.println("hi");
        System.out.println("hi");
    }

}

package Loop;

public class Loop {
    public static void main(String[]args) {

        //repeat this statement 4time
        // loop - while loop,do while loop,For loop

        // 100 - condition
        System.out.println("hi");
    }

}

17. While Loop in java

    While Loop:-while loop is a pre-test loop,ti is used when we dont know the no. of
    iterations in advance.
             It is also known as entry control loop.

    package WhileLoop;

public class WhileLoop {
    public static void main(String[]args) {

        int i = 1;

        while(i<=6)
        {
```

```
            System.out.println("hi"+ i);

            i++;
        }
    }

}

## package WhileLoop;

public class WhileLoop {
    public static void main(String[]args) {

        int i = 1;

        while(i<= 6)
        {
            System.out.println("hi"+ i);

            i++;
        }
        System.out.println("Bye" + i);
    }

}


    ## package WhileLoop;

public class WhileLoop {
    public static void main(String[]args) {

        int i = 1;

        while(i<=4)
        {
            System.out.println("hi"+ i);

            int j = 1;

            while(j<=3) {
                System.out.println("Hello" +j);
                j++;
            }


            i++;
        }
        System.out.println("Bye" + i);
    }

}


  18. Do-While Loop in java

      Do-While Loop:-is a post-test loop,it is used when we want to execute loop body
      atlest once even condition is false.
                          It is also known as exit control loop.
   package DoWhile;

public class DoWhile {
    public static void main(String[]args) {


        int i = 1;

```

```
399          while(i<=4)
400          {
401
402              System.out.println("Hello" +i);
403
404              i++;
405          }
406      }
407
408  }
409
410
411  package DoWhile;
412
413  public class DoWhile {
414      public static void main(String[]args) {
415
416
417          int i = 1;
418
419          do
420          {
421
422              System.out.println("Hello" +i);
423
424              i++;
425          }while(i<=4);
426          }
427      }
428
429
430  19.For-Loop
431
432  package ForLoop;
433
434  public class ForLoop {
435      public static void main(String[]args) {
436
437
438
439          for(int i=1; i<=4;i++) // increment
440          {
441              System.out.println("hi"  +i);
442          }
443
444      }
445
446  }
447
448  ## package Array;
449
450  public class Array {
451
452      public static void main(String[]args)
453      {
454          int i, j;
455          for(i=1;i<=5; i++)
456          {
457              for(j=1;j<=i;j++)
458              {
459                  System.out.print(" * ");
460              }
461              System.out.print("\n");
462          }
463          }
464      }
465
```

```
package ForLoop;

public class ForLoop {
    public static void main(String[]args) {

        //int i = 1;

        for(int i=4; i>=1; i--) //decrement
        {
            System.out.println("hi"  +i);
        }

    }

}

## package ForLoop;

public class ForLoop {
    public static void main(String[]args) {

        //int i = 1;

        for(int i=0; i<=3; i++)
        {
            System.out.println("hi"  +i);
        }

    }

}

## less than
package ForLoop;

public class ForLoop {
    public static void main(String[]args) {

        //int i = 1;

        for(int i=0; i<4; i++)
        {
            System.out.println("hi"  +i);
        }

    }

}

## package ForLoop;

public class ForLoop {
    public static void main(String[]args) {

        //int i = 1;

        for(int i=1; i<=7; i++)
        {
            System.out.println("Day"  +i);
        }

    }

}
```

```
## package ForLoop;

public class ForLoop {
    public static void main(String[]args) {

        //int i = 1;

        for(int i=1; i<=5; i++)
        {
            System.out.println("Day"  +i);

            for(int j=1; j<=9; j++)
            {



            System.out.println("   9");
            }
        }

    }

}


## package ForLoop;

public class ForLoop {
    public static void main(String[]args) {

        //int i = 1;

        for(int i=1; i<=5; i++)
        {
            System.out.println("Day"  +i);

            for(int j=1; j<=9; j++)
            {



            System.out.println("  " +j);
            }
        }

    }

}

## package ForLoop;

public class ForLoop {
    public static void main(String[]args) {

        //int i = 1;

        for(int i=1; i<=5; i++)
        {
            System.out.println("Day"  +i);

            for(int j=1; j<=9; j++)
            {



            System.out.println("  " +(j+8));
            }
```

```
        }

    }

}

## package ForLoop;

public class ForLoop {
    public static void main(String[]args) {

        //int i = 1;

        for(int i=1; i<=5; i++)
        {
            System.out.println("Day"  +i);

            for(int j=1; j<=9; j++)
            {



            System.out.println("  " +(j+8) + " - " + (j+9));
            }
        }

    }

}


## package ForLoop;

public class ForLoop {
    public static void main(String[]args) {

        int i = 1;

        for(; i<=5;)
        {
            System.out.println("Day"  +i);

        //  for(int j=1; j<=9; j++)
        //  {



        //  System.out.println("  " +(j+8) + " - " + (j+9));
        //  }
            i++;
        }

    }

}
```

21.Class And Object Theory in java

Class:- class is a collection of objects and it doesnt take any space on memory,class is also called as bluprint/ logical entity.

Object:-object is an instace of class that execute the class. onec the object is created, it takes up space like other variable in momory.

syntax:- class-name obj-name = new class-name

```
//  Object Oriented Programing

  // Object- Properties and Behaviors

  first create:----
  // class
  give the design in class file you have a class files which a buluprint



  22.Class and Object Practical in java

  package Object;

public class Object {

    public static void main(String[]args)
    {

    int num1 = 4;
    int num2 = 5;

    int result = num1 + num2;

    System.out.println(result);


    }

}
 ## package Object;

public class Eample {

    public static void main(String[]args) {
        System.out.println("Hello class");

    }

    public static void get(String[]args) {
         System.out.println("Hello class");
    }

}

## package Object;

public class Eample {



    public static void main(String[]args) {
        System.out.println("Hello Mantu");

    }

    }


 class User{
    public int data =10;
    public  String name ="Mantu";

    public static void get(String[]args) {
         System.out.println("Hello class");
    }

```

```
}


  ## package Object;

public class Eample {



    public static void main(String[]args) {
        User U1 = new User();
        System.out.println("Hello ");

    }

    }


 class User{
    public int data =10;
    public  String name ="Mantu";

    public static void get(String[]args) {
         System.out.println("Hello class");
    }

}

## package Object;


class demo{
    int a ;

    public int add()
    {
         System.out.println("in add");
         return 0;
    }

}



 public class Eample
 {
     public static void main(String[]args) {

         int num1=4;
         int num2=5;
         int result = num1 + num2;
         System.out.println(result);

    }







    }


23.JDK JRE JVM in java
```

24.Method in java

```
package Calculator;


class Computer
{

    public void playMusic()
    {
        System.out.println("Music playing...");
    }

    public String  getMeAPen(int cost)
    {
        return "Pen";
    }
    }

public class Calculator {

    public static void main(String[]args) {

    Computer obj = new Computer();
    obj.playMusic();
    }
}


## package Calculator;


class Computer
{

    public void playMusic()
    {
        System.out.println("Music playing...");
    }

    public String  getMeAPen(int cost)
    {
        return "Pen";
    }
    }

public class Calculator {

    public static void main(String[]args) {

    Computer obj = new Computer();
    obj.playMusic();

    String str =obj.getMeAPen( 10);
    System.out.println(str);
    }
}


## package Calculator;


class Computer
{
```

```
    public void playMusic()
    {
        System.out.println("Music playing...");
    }

    public String  getMeAPen(int cost)
    {
        if(cost >= 10)
        return "Pen";
        else
            return "Nothing";
    }
    }
public class Calculator {

    public static void main(String[]args) {

    Computer obj = new Computer();
    obj.playMusic();

    String str =obj.getMeAPen( 2);
    System.out.println(str);
    }
}


## package Calculator;


class Computer
{

    public void playMusic()
    {
        System.out.println("Music playing...");
    }

    public String  getMeAPen(int cost)
    {
        if(cost >= 10)
        return "Pen";

            return "Nothing";
    }
    }

public class Calculator {

    public static void main(String[]args) {

    Computer obj = new Computer();
    obj.playMusic();

    String str =obj.getMeAPen( 12);
    System.out.println(str);
    }
}



25. Method Overloding in java

package Calculator;
```

```
class School
{
    public int add(int n1, int n2)
    {

        return n1 + n2;
    }

    }



public class Calculator {

    public static void main(String[]args) {
        School obj = new School();
        int r1 = obj.add ( 3, 4);
        System.out.println(r1);
    }

    }


## Three nubmer add

package Calculator;


class School
{
    public int add(int n1, int n2 ,int n3)
    {

        return n1 +   n2 + n3 ;
    }

    }



public class Calculator {

    public static void main(String[]args) {
        School obj = new School();
        int r1 = obj.add ( 3, 4 , 9);
        System.out.println(r1);
    }

    }

##  package Calculator;


class School
{
    public int add(int n1, int n2 ,int n3)
    {

        return n1 +   n2 + n3 ;
    }

    public int  add(int n1, int n2)
    {
        return n1+ n2;
    }
    public double  add(double n1, int n2)
    {
```

```
        return n1 + n2;
    }
    }



public class Calculator {

    public static void main(String[]args) {
        School obj = new School();
        int r1 = obj.add ( 3, 4 );
        System.out.println(r1);
    }

    }


26. Stack And Heap in java?

package Calculator;

class School
{
    int num;
    public int add(int n1, int n2 )
    {

        return n1 +   n2  ;
    }

    }

public class Calculator {

    public static void main(String[]args) {
        int data = 10;

        School obj = new School();
        int r1 = obj.add ( 3, 4 );
        System.out.println(r1);
    }

    }

## package Calculator;


class School
{
    int num = 5;
    public int add(int n1, int n2 )
    {
        System.out.println(num);
        return n1 +   n2  ;
    }

    }
public class Calculator {

    public static void main(String[]args) {
        int data = 10;

        School obj = new School();
        int r1 = obj.add ( 3, 4 );
        System.out.println(r1);
    }

```

```
    }

## package Calculator;


class School
{
    int num=5;
    public int add(int n1, int n2 )
    {
    //  System.out.println(num);
        return n1 +   n2  ;
    }

    }

public class Calculator {

    public static void main(String[]args) {
        int data = 10;

        School obj = new School();
        School obj1 = new School();
        int r1 = obj.add ( 3, 4 );
    //  System.out.println(r1);
        System.out.println(obj.num);
        System.out.println(obj1.num);
    }

    }


    ## package Calculator;


class School
{
    int num=5;
    public int add(int n1, int n2 )
    {
    //  System.out.println(num);
        return n1 +   n2  ;
    }

    }



public class Calculator {

    public static void main(String[]args) {
        int data = 10;

        School obj = new School();
        School obj1 = new School();
        int r1 = obj.add ( 3, 4 );
    //  System.out.println(r1);

        obj.num = 8;

        System.out.println(obj.num);
        System.out.println(obj1.num);
    }

    }

27. Need of an Array in java
```

```
Array:- in a variable can store an value type:-
Integer,Falot,Double,String,Boolean

example:- int i = 5;
          int j = 6;
          int k = 7;

          int num [] ={5,6,7};

          int num1[] = new int[4];

          Index number
          [3][7][2][4]
          0   1  2  3


        ##  package Array;

public class Array {

    public static void main(String[]args)
    {
        int nums[] = {3,7,4};
        System.out.println(nums[0]);
    }

}

## package Array;

public class Array {

    public static void main(String[]args)
    {
        int nums[] = {3,7,4, 9};
        System.out.println(nums[1]);
    }

}

## package Array;

public class Array {

    public static void main(String[]args)
    {
        int nums[] = {3,7,4, 9};
        nums[1]=6;



        System.out.println(nums[1]);
    }

}

## package Array;

public class Array {

    public static void main(String[]args)
    {
        int nums[] = new int[4];
    //  nums[1]=6;
```

```
        System.out.println(nums[1]);
    }

}
// index no:-0

## package Array;

public class Array {

    public static void main(String[]args)
    {
        int nums[] = new int[4];
        nums[0]=4;
        nums[1]=8;
        nums[2]=3;
        nums[3]=9;

        System.out.println(nums[0]);
        System.out.println(nums[1]);
        System.out.println(nums[2]);
        System.out.println(nums[3]);
    }

}

## package Array;

public class Array {

    public static void main(String[]args)
    {
        int nums[] = new int[4];
        nums[0]=4;
        nums[1]=8;
        nums[2]=3;
        nums[3]=9;

        for(int i =0; i<=3; i++)
        {
            System.out.println(nums[i]);
        }

    }
}


29.Multi Dimensional Array in java

package Array;

public class Array {

    public static void main(String[]args)
    {
        int nums[][] = new int[3][4];

        for(int i=0;i<3;i++)
        {

            for(int j=0;j<4;j++)
            {
                System.out.print(nums[i][j] +  " ");
            }
            System.out.println();
        }
```

```
    }
}

## package Array;

public class Array {

    public static void main(String[]args)
    {
        int nums[][] = new int[3][4];


        for(int i=0;i<3;i++)
        {

            for(int j=0;j<4;j++)
            {
                nums[i][j] = (int)(Math.random() *100);
                System.out.println(nums[i][j]);
            }

        }




        for(int i=0;i<3;i++)
        {

            for(int j=0;j<4;j++)
            {
                System.out.print(nums[i][j] +  " ");
            }
            System.out.println();
        }


    }
}

## package Array;

public class Array {

    public static void main(String[]args)
    {
        int nums[][] = new int[3][4];


        for(int i=0;i<3;i++)
        {

            for(int j=0;j<4;j++)
            {
                nums[i][j] = (int)(Math.random() *100);

            }

        }




        for(int i=0;i<3;i++)
        {
```

```
            for(int j=0;j<4;j++)
            {
                System.out.print(nums[i][j] +  " ");
            }
            System.out.println();
        }


    }
}

## package Array;

public class Array {

    public static void main(String[]args)
    {
        int nums[][] = new int[3][4];


        for(int i=0;i<3;i++)
        {

            for(int j=0;j<4;j++)
            {
                nums[i][j] = (int)(Math.random() *10);

            }

        }




        for(int i=0;i<3;i++)
        {

            for(int j=0;j<4;j++)
            {
                System.out.print(nums[i][j] +  " ");
            }
            System.out.println();
        }


    }
}


## package Array;

public class Array {

    public static void main(String[]args)
    {
        int nums[][] = new int[3][4];


        for(int i=0;i<3;i++)
        {

            for(int j=0;j<4;j++)
            {
                nums[i][j] = (int)(Math.random() *10);

            }
```

```
        }



        for(int i=0;i<3;i++)
        {

            for(int j=0;j<4;j++)
            {
                System.out.print(nums[i][j] +" ");
            }
            System.out.println();
        }

        for(int n[] : nums)
        {
            for(int m : n)
            {
                System.out.println(m+ "");
            }
            System.out.println();
        }


    }
}

 ## package Array;

public class Array {

    public static void main(String[]args)
    {
        int nums[][] = new int[3][4]; // jagged Array

        nums[0]= new int[3];
        nums[1]= new int[4];
        nums[2]= new int[2];

        for(int i=0;i<nums.length;i++)
        {

            for(int j=0;j<nums[i].length;j++)
            {
                nums[i][j] = (int)(Math.random() *10);

            }

        }



        for(int n[]: nums)
        {
            for(int m: n)
            {
                System.out.print(m + " ");
            }
            System.out.println();

        }
    }
    }

31.Drawbacks of Array in java
```

```
package Array;

public class Array
{

    public static void main(String[]args)
    {

        int nums[] = new int[4];
        nums[0] = 4;
        nums[1] = 8;
        nums[2] = 3;
        nums[3] = 9;

        for(int i=0;i<4;i++)
        {
            System.out.println(nums[i]);
        }
    }
    }


32.Array of Object in java

package Array;

public class Array
{

    public static void main(String[]args)
    {

        int nums[] = new int[6];
        nums[0] = 4;
        nums[1] = 8;
        nums[2] = 3;
        nums[3] = 9;

        for(int i=0;i<6;i++)
        {
            System.out.println(nums[i]);
        }
    }
    }

## package Array;

public class Array
{

    public static void main(String[]args)
    {

        int nums[] = new int[6];
        nums[0] = 4;
        nums[1] = 8;
        nums[2] = 3;
        nums[3] = 9;

        for(int i=0;i<nums.length;i++)
        {
            System.out.println(nums[i]);
        }
    }
    }

## package Array;
```

```
class Students
{
    int rollno;
    String name;
    int marks;
}

public class Array
{

    public static void main(String[]args)
    {
        Students s1 = new Students();
        s1.rollno = 1;
        s1.name = "Mantu";
        s1.marks = 96;

        Students s2 = new Students();
        s2.rollno = 2;
        s2.name = "Karan";
        s2.marks = 94;

        Students s3 = new Students();
        s3.rollno = 3;
        s3.name = "Aman";
        s3.marks = 88;

        System.out.println(s2.name + " : " + s2.marks);

        Students students[] = new Students[3];
        students[0] = s1;
        students[1] = s2;
        students[2] = s3;


    }
    }
```

33.Enhanced for Loop in java

```
package Array;


public class Array
{

    public static void main(String[]args)
    {
        int nums[] = new int [4];

            nums[0] = 4;
            nums[1] = 8;
            nums[2] = 3;
            nums[3] = 9;

            for(int n : nums)
            {
                System.out.println(n);
            }
        }
    }
```

34.What is String in java

```
package Array;


public class Array
{

    public static void main(String[]args)
    {
        String name = new String( "Mantu");

        System.out.println(name);
        }
    }

## package Array;


public class Array
{

    public static void main(String[]args)
    {
        String name = new String( "mantu");

        System.out.println(name);
        System.out.println(name.hashCode());
        }
    }

## package Array;


public class Array
{

    public static void main(String[]args)
    {
        String name = new String( "mantu");

        System.out.println("hello" + name );

        }
    }

## package Array;


public class Array
{

    public static void main(String[]args)
    {
        String name = new String( "mantu");

        System.out.println("hello" + name );
        System.out.println(name.charAt(1));
        }
    }

## package Array;


public class Array
{

    public static void main(String[]args)
    {
```

```
1670          String name = new String( "mantu");
1671
1672          System.out.println("hello" + name );
1673          System.out.println(name.concat("kumar"));
1674          }
1675      }
1676
1677  ## package Array;
1678
1679
1680  public class Array
1681  {
1682
1683      public static void main(String[]args)
1684      {
1685          String name = "mantu";
1686
1687          System.out.println("hello" + name );
1688
1689          }
1690      }
1691
1692      35.Mutable vs immutable String in java
1693           package Array;
1694
1695
1696  public class Array
1697  {
1698
1699      public static void main(String[]args)
1700      {
1701          String name = " mantu";
1702          name = name+ " kumar ";
1703          System.out.println(" hello" + name );
1704
1705          }
1706      }
1707
1708
1709  36.StringBuffer and StringBuilder in java
1710
1711  . An objects of string class are immutable, objects of
1712    stringBuffer and stringBuilder class are mutable.
1713
1714
1715
1716
1717      package Array;
1718
1719
1720  public class Array
1721  {
1722
1723      public static void main(String[]args)
1724      {
1725          StringBuffer sb = new StringBuffer("Mantu");
1726          System.out.println(sb.capacity());
1727          }
1728      }
1729
1730
1731  ## package Array;
1732
1733
1734  public class Array
1735  {
1736
```

```
    public static void main(String[]args)
    {
        StringBuffer sb = new StringBuffer("Mantu");
        System.out.println(sb.length());
        }
    }


## package Array;


public class Array
{

    public static void main(String[]args)
    {
        StringBuffer sb = new StringBuffer("Mantu");
        sb.append("kumar");

        System.out.println(sb);
        }
    }

## package Array;


public class Array
{

    public static void main(String[]args)
    {
        StringBuffer sb = new StringBuffer("Mantu");
        sb.append("kumar");

        sb.deleteCharAt(2);  // deleteCharAt

        System.out.println(sb);


        }
    }


    ## package Array;


public class Array
{

    public static void main(String[]args)
    {
        StringBuffer sb = new StringBuffer("Mantu");
        sb.append("kumar");

        sb.insert(0, "java  "); // insert

        System.out.println(sb);
        }

        }


    ## package Array;


public class Array
{
```

```
    public static void main(String[]args)
    {
        StringBuffer sb = new StringBuffer("Mantu");
        sb.append("kumar");

        sb.insert(6, "java  ");
        sb.setLength(30);


        System.out.println(sb);


        }
    }
```

37. String Variable in java

```
package Mantu;

class Mobile
{
    String brand;
    int price;
    String name;

    public void show()
    {
        System.out.println(brand +": "+ price +": "+ name);
    }
    }

public class Mantu {
public static void main(String[]args)
{
    Mobile obj1 = new Mobile();
    obj1.brand = "Apple";
    obj1.price = 1500;
    obj1.name = "Smart Phone";

    Mobile obj2 = new Mobile();
    obj2.brand = "Nokia";
    obj2.price = 1200;
    obj2.name = "Smart Phone";

    obj1.show();
    obj2.show();
    }
}


## CHANGE THE NAME:-
package Mantu;

class Mobile
{
    String brand;
    int price;
    static String name;

    public void show()
    {
        System.out.println(brand +": "+ price +": "+ name);
    }
    }
```

```
1871
1872        Mobile obj1 = new Mobile();
1873        obj1.brand = "Apple";
1874        obj1.price = 1500;
1875        obj1.name = "Smart Phone";
1876
1877        Mobile obj2 = new Mobile();
1878        obj2.brand = "Nokia";
1879        obj2.price = 1200;
1880        obj2.name = "Smart Phone";
1881
1882        obj1.name = "Phone";
1883
1884        obj1.show();
1885        obj2.show();
1886        }
1887    }
1888
1889    public class Mantu {
1890    public static void main(String[]args)
1891    {
1892     38.Static Method in java
1893     package Mantu;
1894
1895    class Mobile
1896    {
1897        String brand;
1898        int price;
1899        static String name;
1900
1901        public void show()
1902        {
1903            System.out.println(brand +": "+ price +": "+ name);
1904        }
1905
1906
1907    public static void show1(Mobile obj)
1908    {
1909        System.out.println(obj.brand + " : " + obj.price +" : " + name);
1910    }
1911        }
1912
1913
1914    public class Mantu {
1915    public static void main(String[]args)
1916    {
1917        Mobile obj1 = new Mobile();
1918        obj1.brand = "Apple";
1919        obj1.price = 1500;
1920        Mobile.name = "Smart Phone";
1921
1922        Mobile obj2 = new Mobile();
1923        obj2.brand = "Nokia";
1924        obj2.price = 1200;
1925        Mobile.name = "Smart Phone";
1926
1927        Mobile.name = "Phone";
1928
1929        obj1.show();
1930        obj2.show();
1931
1932        Mobile.show1(obj1);
1933        }
1934    }
1935
1936     39.Static Block in java
1937     package Mantu;
```

```
class Mobile
{
    String brand;
    int price;
    static String name;  // initialization


    public Mobile()
    {
        brand = "";
        price = 2000;
           System.out.println("in constructor");            // constructor
        }
    static
    {
        name = "Phone";
        System.out.println("in static block");
    }

    public void show()
    {
        System.out.println(brand +": "+ price +": "+ name);
    }

}

public class Mantu {
public static void main(String[]args)
{
    Mobile obj1 = new Mobile();
    obj1.brand = "Apple";
    obj1.price = 1500;
    Mobile.name = "Smart Phone";

    Mobile obj2 = new Mobile();
    obj2.brand = "Nokia";
    obj2.price = 1200;
    Mobile.name = "Smart Phone";

    Mobile.name = "Phone";




    }
}
```

40.Encapsulation in java

```
package Mantu;

class Human
{
    int age;
    String name;
}
public class Mantu {

public static void main(String[]args)
{
    Human obj = new Human();
    obj.age = 15;
    obj.name = "Mantu";

    System.out.println(obj.name);
}
```

```
}


## package Mantu;

class Human
{
    private int age = 11;
   private String name = "Mantu";

   public int getAge()
   {
       return age;
   }

   public String getName()
   {
       return name;
   }
}
public class Mantu {

public static void main(String[]args)
{
    Human obj = new Human();
    //obj.age = 15;
//  obj.name = "Mantu";

    System.out.println(obj.getName() + " : " + obj.getAge());
}

}

## package Mantu;

class Human
{
    private int age;
   private String name;

   public int getAge()
   {
       return age;
   }

   public String getName()
   {
       return name;
   }
}
public class Mantu {

public static void main(String[]args)
{
    Human obj = new Human();
    //obj.age = 15;
//  obj.name = "Mantu";

    System.out.println(obj.getName() + " : " + obj.getAge());
}

}

## package Mantu;

class Human
```

```
{
    private int age;
   private String name;

   public int getAge()
   {
       return age;
   }
   public void setAge(int a)
   {
       age = a;
   }
   public String getName()
   {
       return name;
   }
   public void setName(String n )
   {
       name = n;
   }
}
public class Mantu {

public static void main(String[]args)
{
    Human obj = new Human();
    obj.setAge(30);
    obj.setName("Mantu");

    System.out.println(obj.getName() + " : " + obj.getAge());
}

}
```

41. Getters and Setters in java

42.This Keyword in java

```
package Mantu;

class Human
{
    private int age;
   private String name;

   public int getAge()
   {
       return age;
   }
   public void setAge(int a)
   {
       age = a;
   }
   public String getName()
   {
       return name;
   }
   public void setName(String n )
   {
       name = n;
   }
}
public class Mantu {

public static void main(String[]args)
{
    Human obj = new Human();
```

```
     obj.setAge(30);
     obj.setName("Mantu");

     System.out.println(obj.getName() + " : " + obj.getAge());
}

}

## package Mantu;

class Human
{
     private int age;
    private String name;

    public int getAge()
    {
        return age;
    }
    public void setAge(int age)
    {
        Human obj1 = new Human();
        obj1.age = age;
    }
    public String getName()
    {
        return name;
    }
    public void setName(String n)
    {
        name = n;
    }
}
public class Mantu {

public static void main(String[]args)
{
     Human obj = new Human();
     obj.setAge(30);
     obj.setName("Mantu");

     System.out.println(obj.getName() + " : " + obj.getAge());
}

}

## package Mantu;

class Human
{
     private int age;
    private String name;

    public int getAge()
    {
        return age;
    }
    public void setAge(int age,Human obj)
    {
        Human obj1 = obj;
        obj1.age = age;
    }
    public String getName()
    {
        return name;
    }
    public void setName(String n)
```

```
    {
        name = n;
    }
}
public class Mantu {

public static void main(String[]args)
{
    Human obj = new Human();
    obj.setAge(30,obj);
    obj.setName("Mantu");

    System.out.println(obj.getName() + " : " + obj.getAge());
}

}
```

43.Constructor in java

```
package Mantu;

class Human
{
    private int age;
   private String name;

   // no -argument constructor
   public Human()
   {
       age = 12;
       name = "Aman";
   }



   public int getAge()
   {
       return age;
   }
   public void setAge(int age)
   {

       this.age = age;
   }
   public String getName()
   {
       return name;
   }
   public void setName(String name)
   {
       this.name = name;
   }
}
public class Mantu {

public static void main(String[]args)
{
    // calling no -argument constructor
    Human obj = new Human();
    Human obj1 = new Human();
    System.out.println(obj.getName() + " : " + obj.getAge());


    //obj.setAge(30);
    //obj.setName("Mantu");

```

```
//  System.out.println(obj.getName() + " : " + obj.getAge());
}

}
```

44. Default vs Parameterized Constructor in java

```
package Mantu;

class Human
{
    private int age;
   private String name;

   // default constructor
   public Human()
   {
       age = 12;
       name = "Aman";
   }
       // parameterized constructor
   public Human(int a, String n)
   {
       age =a;
       name = n;
   }

   public int getAge()
   {
       return age;
   }
   public void setAge(int age)
   {

       this.age = age;
   }
   public String getName()
   {
       return name;
   }
   public void setName(String name)
   {
       this.name = name;
   }
}
public class Mantu {

public static void main(String[]args)
{
    // calling no -argument constructor
    Human obj = new Human();
    Human obj1 = new Human(18, "Mantu");
    System.out.println(obj.getName() + " : " + obj.getAge());
    System.out.println(obj1.getName() + " : " + obj1.getAge());


    //obj.setAge(30);
    //obj.setName("Mantu");

//  System.out.println(obj.getName() + " : " + obj.getAge());
}

}
```

45. Naming Convention in java

```
 // Camel Casing
```

```
 // class and interface is capital letter
 Example:-Calculator:-Calculator
 // variable and method smoll letter
 Example:- marks,show()
 //Constants:- PIE, BRAND
 // ShowMyMarks()
 //MyDate
 // age, DATA, Human()


46.Anonymous Object in java
package Mantu;

class A
{
    public A()
    {
        System.out.println("object.created");
        }
   public void show()
   {
       System.out.println(" in A show");
       }

   }

public class Mantu
{
    public static void main(String[]args)
    {
        A obj = new A();
        obj.show();
    }
    }

    ## package Mantu;

class A
{
    public A()
    {
        System.out.println("object.created");
        }
   public void show()
   {
       System.out.println(" in A show");
       }

   }

public class Mantu
{
    public static void main(String[]args)
    {

        new A(); // anonymous object


    }
}

## package Mantu;

class A
{
    public A()
    {
```

```
          System.out.println("object.created");
          }
    public void show()
    {
        System.out.println(" in A show");
        }

    }

public class Mantu
{
    public static void main(String[]args)
    {

        new A().show(); // anonymous object


    }
}

## package Mantu;

class A
{
    public A()
    {
        System.out.println("object.created");
        }
    public void show()
    {
        System.out.println(" in A show");
        }

    }

public class Mantu
{
    public static void main(String[]args)
    {

        new A().show(); // anonymous object
        new A().show();
        // created two objects
    }
}

47.Need of Inheritance in java

// Parent super class baise
{ class Calc
   {

   {{

   add()                  // Child sub draive class
   sub()                       clss Adv Calc
   multi()                     {
   div()                        {{
   {
   }                             }


 48.What is Inheritance in java

package Mantu;

class Calc
```

```
{
    public int add(int n1, int n2)
    {
        return n1 + n2;
    }
    public int sub(int n1, int n2)
    {
        return n1 - n2;
    }
    }



class Mantu{

    public static void main(String[]args)
    {
        Calc obj = new Calc();
        int r1 = obj.add(4, 5);
        int r2 = obj.sub(7, 3);

        System.out.println(r1 + " " + r2);
    }
}
```

49.Single and Multilevel Inheritance in java

```
package VeryAdvancCalc;

  class   Calc
{
public double  power(int n1, int n2 )
{
    return Math.pow(n1, n2);
    }
   public int div(int n1, int n2)
   {
       return n1/n2;
   }
}
 public class VeryAdvancCalc extends Calc{

    public static void main(String[]args)
    {
        VeryAdvancCalc obj = new VeryAdvancCalc();
        //int r1 = obj.Add(4,5);
        //int r2 = obj.sub(7, 3);
        //int r3 = obj.Multi(8,3);
        int r4 = obj.div(8,4);
        double r5 = obj.power(2,3);

        System.out.println( r5 +" ");
    }
 }
```

50.Multiple Inheritance in java:-

java multiple inheritance does not work

```
package VeryAdvancCalc;

 class A{

 }
 class B extends A {

 }
 class C{

```

```
}
public class VeryAdvancCalc extends Calc{

   public static void main(String[]args)
   {
       VeryAdvancCalc obj = new VeryAdvancCalc();
       //int r1 = obj.Add(4,5);
       //int r2 = obj.sub(7, 3);
       //int r3 = obj.Multi(8,3);
       int r4 = obj.div(8,4);
       double r5 = obj.power(4,3);

       System.out.println( r5 +" ");
   }
}
```

51. This and Super Method in java

```
package VeryAdvancCalc;

class A
{

}
class B extends A
{
   public B()
   {
       System.out.println("in B");
   }
}
public class VeryAdvancCalc{

   public static void main(String[]args)
   {
       B obj = new B();
   }
}
```

```
##. package VeryAdvancCalc;

class A
{
    public A() // Constructor
    {
        System.out.println("in A");
    }
}
class B extends A
{
   public B() // Constructor
   {
       System.out.println("in B");
   }
}
public class VeryAdvancCalc{

   public static void main(String[]args)
   {
       B obj = new B();
   }
}


## package VeryAdvancCalc;

class A
```

```
{
    public A() // Constructor
    {
        System.out.println("in A");
    }
}
class B extends A
{
   public B() // Constructor
   {
       System.out.println("in B");
   }
   public B(int n)
   {
       System.out.println("int B int ");
   }
}
public class VeryAdvancCalc{

   public static void main(String[]args)
   {
       B obj = new B();
   }
}

## package VeryAdvancCalc;

class A
{
    public A() // Constructor
    {
        System.out.println("in A");
    }
}
class B extends A
{
   public B() // Constructor
   {
       System.out.println("in B");
   }
   public B(int n)
   {
       System.out.println("int B int ");
   }
}
public class VeryAdvancCalc{

   public static void main(String[]args)
   {
       B obj = new B(5);
   }
}

NOTE:- every constructor in java has a method which a
there menstion that method super();

package VeryAdvancCalc;

class A
{
    public A() // Constructor
    {
        super();
        System.out.println("in A");
    }
    public A(int n)
    {
```

```
         super();
         System.out.println("in A int" );
     }
}
class B extends A
{
   public B() // Constructor
   {
       super();
       System.out.println("in B");
   }
   public B(int n)
   {
       super(n);
       System.out.println("int B int ");
   }
}
public class VeryAdvancCalc{

   public static void main(String[]args)
   {
       B obj = new B(5);
   }
}

## package VeryAdvancCalc;

class A
{
    public A() // Constructor
    {
        super();
        System.out.println("in A");
    }
    public A(int n)
    {
        super();
        System.out.println("in A int" );
    }
}
class B extends A
{
   public B() // Constructor
   {
       super(5);
       System.out.println("in B");
   }
   public B(int n)
   {
       super(n);
       System.out.println("int B int ");
   }
}
public class VeryAdvancCalc{

   public static void main(String[]args)
   {
       B obj = new B();
   }
}

Note:-every class in java extends  the object class
in java


52. Method Overloding in java.

```

```
 package VeryAdvancCalc;

 class A
 {
     public void show()
     {
         System.out.println("in show");
     }
 }
 public class VeryAdvancCalc{

    public static void main(String[]args)
    {
         A obj = new A();
         obj.show();
    }

 ## package Sky;
class A
{
    public void show()
    {
        System.out.println("in show");

    }

    }
      class B extends A
      {

      }

public class Sky {
    public static void main(String[]args)
    {
        B obj = new B();
        obj.show();
    }

}

## package Sky;
class A
{
    public void show()
    {
        System.out.println("in A show");

    }
    public void config()
    {
        System.out.println("in A config");
    }
    }
      class B extends A
      {

      }

public class Sky {
    public static void main(String[]args)
    {
        B obj = new B();
        obj.show();
        obj.config();
    }

```

```
## package Sky;
class A
{
    public void show()
    {
        System.out.println("in A show");

    }
    public void config()
    {
        System.out.println("in A config");
    }
    }
      class B extends A
      {
          public void show() // overriding in a class
          {
              System.out.println("in B show");
          }
      }

public class Sky {
    public static void main(String[]args)
    {
        B obj = new B();
        obj.show();
        obj.config();
    }

}

## package Sky;
class A
{
    public void show()
    {
        System.out.println("in A show");

    }
    public void config()
    {
        System.out.println("in A config");
    }
    }
      class B extends A
      {
          public void show1() // overriding in a class
          {
              System.out.println("in B show");
          }
      }

public class Sky {
    public static void main(String[]args)
    {
        B obj = new B();
        obj.show1();
        obj.config();
    }

}

## package Sky;
class Calc
{
    public int add(int n1, int n2)
```

```
    {
        return n1+n2;

    }

    }
      class AdvCalc extends Calc
      {
        //  public int add(int n1, int n2) // overriding in a class
         // {
            // return n1+n2+1;
        //  }
      }

public class Sky {
    public static void main(String[]args)
    {
    AdvCalc obj = new AdvCalc();
    int r1=obj.add(3,4);
    System.out.println(r1);
    }

## package Sky;
class Calc
{
    public int add(int n1, int n2)
    {
        return n1+n2;

    }

    }
      class AdvCalc extends Calc
      {
          public int add(int n1, int n2) // This method is  overriding in a class
          {
             return n1+n2+1;
          }
      }

public class Sky {
    public static void main(String[]args)
    {
    AdvCalc obj = new AdvCalc();
    int r1=obj.add(3,4);
    System.out.println(r1);
    }

}



55. Polymorphim in java
    many behaviour

There are two types polymorphim
(1) Compile time :-behaviour is your difinde  compile time.
(overloding):-part of compile time because at the time itself.
(2) Run time :- it is behaviour will be run time.(overriding):-

compile time is two method.

add(int,int)
add(int,int,int) voerloding

                        A
                            add(int,int)
```

B
add(int,int)

56.Dynamic Method Dispatch in java.

```
package Sky;

class A   // parents
{
public void show()
{
    System.out.println("in A show");
    }
    }

 class B extends A  // child
 {

 }

public class Sky {
    public static void main(String[]args)
    {
      B obj = new B();
      obj.show();
    }

}

## package Sky;


class A   // parents
{
public void show()
{
    System.out.println("in A show");
    }
    }

 class B extends A  // child
 {
     public void show()
     {
        System.out.println("in B show");
        }
 }

public class Sky {
    public static void main(String[]args)
    {
      A obj = new B();
      obj.show();


    }

}

##package Sky;


class A   // parents
```

```
{
public void show()
{
    System.out.println("in A show");
    }
    }

 class B extends A  // child
 {
     public void show()
     {
        System.out.println("in B show");
        }
 }

public class Sky {
    public static void main(String[]args)
    {
      A obj = new A();
      obj.show();

     obj = new B();
     obj.show();
    }

}

## package Sky;


class A   // parents
{
public void show()
{
    System.out.println("in A show");
    }
    }

 class B extends A  // child
 {
     public void show()
     {
        System.out.println("in B show");
        }
 }
 class C extends A
 {
     public void show()
     {
        System.out.println("in C show");
        }
 }


public class Sky {
    public static void main(String[]args)
    {
      A obj = new A();
      obj.show();

     obj = new B();
     obj.show();

     obj = new C();
     obj.show();
    }

```

```
}

57. Final Keyword in java
Final Keyword:-can be used to with variable, method, class

package Sky;



public class Sky {
    public static void main(String[]args)
    {

        int num = 8;
        num = 9;
        System.out.println(num);

    }

}

## package Sky;

class Calc
{
    public void show()
    {
        System.out.println("in Calc show");
    }
    public void add(int a,int b)
    {
        System.out.println(a+b);
    }
    }


public class Sky {
    public static void main(String[]args)
    {
     Calc obj = new Calc();
     obj.show();
     obj.add(8, 12);
    }

}


## package Sky;

 class Calc
{
    public void show()
    {
        System.out.println("in Calc show");
    }
    public void add(int a,int b)
    {
        System.out.println(a+b);
    }
    }

class AdvCalc extends Calc
{

    }

public class Sky {
```

```
    public static void main(String[]args)
    {
     AdvCalc obj = new AdvCalc();
     obj.show();
     obj.add(8, 12);
    }

}

## package Sky;

 class Calc
{
    public void show()
    {
        System.out.println("By Mantu");
    }
    public void add(int a,int b)
    {
        System.out.println(a+b);
    }
    }

class AdvCalc extends Calc
{
    public void show()
    {
        System.out.println("By puja"); // Overriding
    }
    }

public class Sky {
    public static void main(String[]args)
    {
     AdvCalc obj = new AdvCalc();
     obj.show();
     obj.add(8, 12);
    }

}


58.Object class equals to String hashCode in java

package Sky;

class Laptop
{
     String model;
     int price;

     public String toString()
     {
         return "Hey";
     }
    }

public class Sky {
    public static void main(String[]args)
    {
     Laptop obj = new Laptop();
     obj.model = "Hp word ";
     obj.price = 50000;

     System.out.println(obj.toString());
    }

```

```
}

## package Sky;

class Laptop
{
     String model;
     int price;

     public String toString()
     {
         return "Hey";
     }
    }

public class Sky {
    public static void main(String[]args)
    {
     Laptop obj = new Laptop();
     obj.model = "Hp word ";
     obj.price = 50000;

     System.out.println(obj);
    }

}

## package Sky;

class Laptop
{
     String model;
     int price;

     public String toString()
     {
         return model + " :" + price;
     }
    }

public class Sky {
    public static void main(String[]args)
    {
     Laptop obj = new Laptop();
     obj.model = "Hp word ";
     obj.price = 50000;

     System.out.println(obj);
    }

}


## package Sky;

class Laptop
{
     String model;
     int price;

     public String toString()
     {
         return model + " :" + price;
     }
    }

public class Sky {
```

```
    public static void main(String[]args)
    {
     Laptop obj1 = new Laptop();
     obj1.model = "Hp word ";
     obj1.price = 50000;

     Laptop obj2 = new Laptop();
     obj2.model = "Hp word ";
     obj2.price = 50000;

    boolean result = obj1 == obj2;

     System.out.println(result);
    }

}

## package Sky;

class Laptop
{
     String model;
     int price;

     public String toString()
     {
         return model + " :" + price;
     }
     public boolean equals(Laptop that)
     {
         if(this.model.equals(that.model) && this.price == that.price)
         {
     return true;
    }

     else

         return false;
    }
}

public class Sky {
    public static void main(String[]args)
    {
     Laptop obj1 = new Laptop();
     obj1.model = "Hp word ";
     obj1.price = 50000;

     Laptop obj2 = new Laptop();
     obj2.model = "Hp word ";
     obj2.price = 50000;

    boolean result = obj1.equals(obj2);

     System.out.println(result);
    }

}

59.Upcasting and Downcasting in java.

package Sky;


public class Sky {
    public static void main(String[]args)
    {
```

```
    double d = 4.5;

    System.out.println(d);
    }

}

## package Sky;


public class Sky {
    public static void main(String[]args)
    {
    double d = 4.5;
    int i = (int) d; // type casting

    System.out.println(i);
    }

}

 ## package Sky;

class A
{
    public void show1()
    {
        System.out.println("in A show");
    }
    }
   class B extends A
   {
     public void show2()
     {
         System.out.println("in B show");
     }
   }


public class Sky {
    public static void main(String[]args)
    {
      A obj = new A();
      obj.show1();
    }


## package Sky;

class A
{
    public void show1()
    {
        System.out.println("in A show");
    }
    }
   class B extends A
   {
     public void show2()
     {
         System.out.println("in B show");
     }
   }


public class Sky {
    public static void main(String[]args)
```

```
    {
      A obj = (A)new B(); // this method is Upcasting
      obj.show1();

    }

}

## package Sky;

class A
{
    public void show1()
    {
        System.out.println("in A show");
    }
    }
   class B extends A
   {
     public void show2()
     {
         System.out.println("in B show");
     }
   }


public class Sky {
    public static void main(String[]args)
    {
      A obj = new B();
      obj.show1();


      B obj1 = (B) obj; // This method is  Downcasting
      obj1.show2();

    }

}

60.Wrapper Class in java.

For every primitive type we are going to have class file
  int- Integer
  char- Charatctor
  double- Double

  package Sky;

public class Sky {
    public static void main(String[]args)
    {
     int num = 7;
     Integer num1 = ( 8); // this called Autoboxing


     System.out.println(num1);
    }

}

## package Sky;

public class Sky {
    public static void main(String[]args)
    {
     int num = 7;
```

```
      Integer num1 = num; // Autoboxing

      int num2 = num1.intValue();  // auto- Unboxing

      System.out.println(num2);
     }

}

## package Sky;

public class Sky {
    public static void main(String[]args)
    {
     int num = 7;
     Integer num1 = num; // Autoboxing


     int num2 = num1.intValue();  // auto- Unboxing

     System.out.println(num2);

     String str = "12";
     int num3 = Integer.parseInt(str);

     System.out.println(num3*2);

    }

}
```

61.Abstract Keyword in java

```
package Sky;


class car
{
    public void drive()
    {

    }
    public void playMusic()
    {
        System.out.println("play music");
    }
    }

##
public class Sky {
    public static void main(String[]args)
    {
     car obj = new car();
     obj.drive();
     obj.playMusic();
    }

}
```

```
public class Sky {
    public static void main(String[]args)
    {
```

```
      int num = 7;
      Integer num1 = new Integer( 8);//worning


      System.out.println(num1);
     }

}

## package Sky;

public class Sky {
    public static void main(String[]args)
    {
      int num = 7;
      Integer num1 = ( 8);


      System.out.println(num1);
     }

}

62. Inner class in java.
  class A
{
    int age ;

    public void show()
    {
        System.out.println("in show");
    }
    class B
    {
        public void config()
        {
            System.out.println("in config");
        }
    }
    }


public class Sky {
    public static void main(String[]args)
    {
        A obj = new A();
        obj.show();

    }
## package Sky;

class A
{
    int age ;

    public void show()
    {
        System.out.println("in show");
    }
    class B
    {
        public void config()
        {
            System.out.println("in config");
        }
    }
    }
```

```
public class Sky {
    public static void main(String[]args)
    {
        A obj = new A();
        obj.show();

        A.B obj1 = obj.new B();
       obj1.config();
    }

}

## package Sky;

class A
{
    int age ;

    public void show()
    {
        System.out.println("in show");
    }
    static class B
    {
        public void config()
        {
            System.out.println("in config");
        }
    }
    }


public class Sky {
    public static void main(String[]args)
    {
        A obj = new A();
        obj.show();

        A.B obj1 = new A.B();
       obj1.config();
    }

}
```

63. Anonymous inner class in java.

```
package Sky;

class A
{
    public void show()
    {
        System.out.println("in show");
    }
    }


public class Sky {
    public static void main(String[]args)
    {
        A obj = new A();
        obj.show();
    }

}
```

```
## package Sky;

class A
{
    public void show()
    {
        System.out.println("in A show");
    }
    }
  class B extends A
  {
      public void show()
      {
          System.out.println("in B show");
      }
  }

public class Sky {
    public static void main(String[]args)
    {
        A obj = new B();
        obj.show();
    }

}
```

64.Abstract and Anonymous inner class.

```
package Sky;

abstract class A
{
    public void show()
    {

    }
}
 class B extends A
 {
     public void show()
     {
         System.out.println("in B show");
     }
 }
public class Sky {
    public static void main(String[]args)
    {
        A obj = new B();
        obj.show();

    }

}

## package Sky;

abstract class A
{
    public abstract void show()
    {

    }
}
public class Sky {
    public static void main(String[]args)
```

```
    {
        A obj = new A()
        {
            public void show()
            {
                System.out.println("in new show");
            }
        };
        obj.show();

    }

}

65.What is Interface in java.
package Sky;

interface A
{
     void show();
     void config();
    }
class B implements A
{
    public void show()
    {
        System.out.println("in show");
    }
    public void config()
    {
        System.out.println("in config");
    }
    }
public class Sky {
    public static void main(String[]args)
    {
        A obj;
        obj =new B();
        obj.show();
        obj.config();
    }

}

## package Sky;

interface A
{
    int age=44; // every veriable in the interface final and static
    String area="Patna";
     void show();
     void config();
    }
class B implements A
{
    public void show()
    {
        System.out.println("in show");
    }
    public void config()
    {
        System.out.println("in config");
    }
    }
public class Sky {
    public static void main(String[]args)
    {
```

```
        A obj;
        obj =new B();
        obj.show();
        obj.config();

        System.out.println(A.area);
    }

}
```

66.Need of Interface in java.

```
package Sky;

class  Developer
{
    public void devApp()
    {
        System.out.println("coding");
    }
    }

public class Sky {
    public static void main(String[]args)
    {
        Developer mantu = new Developer();
        mantu.devApp();
    }

}

## package Sky;

class Laptop
{
    public void code()
    {
        System.out.println("code, compile, run");
    }
    }



class  Developer
{
    public void devApp(Laptop lap)
    {
        lap.code();
    }
    }

public class Sky {
    public static void main(String[]args)
    {
        Laptop lap = new Laptop();



        Developer mantu = new Developer();
        mantu.devApp(lap);
    }

}
```

67.

68.What is Enum in java.

```
 package Sky;

enum Status
{
    Running,Failed,Pending,Success;
    }

public class Sky {
    public static void main(String[]args)
    {
        int i = 5;
        Status s = Status.Running;
        System.out.println(s);
    }

}
## package Sky;

enum Status
{
    Running,Failed,Pending,Success;
    }

public class Sky {
    public static void main(String[]args)
    {
        int i = 5;
        Status s = Status.Failed;
        System.out.println(s);
    }

}

##package Sky;

enum Status
{
    Running,Failed,Pending,Success;
    }

public class Sky {
    public static void main(String[]args)
    {
        int i = 5;
        Status s = Status.Running;
        System.out.println(s.ordinal());
    }

}

## package Sky;

enum Status
{
    Running,Failed,Pending,Success;
    }

public class Sky {
    public static void main(String[]args)
    {
        int i = 5;
        Status[] ss = Status.values();
        System.out.println(ss[0]);
    }

}
## package Sky;
```

```
enum Status
{
    Running,Failed,Pending,Success;
    }

public class Sky {
    public static void main(String[]args)
    {
        int i = 5;
        Status[] ss = Status.values();

        for(Status s: ss)
        {
            System.out.println(s);
        }
    }

}
```

69.Enum if and Switch in java.

```
package Sky;

enum Status
{
    Running,Failed,Pending,Success;
    }

public class Sky {
    public static void main(String[]args)
    {
        Status s = Status.Pending;

        if(s == Status.Running)
            System.out.println("All Good");
        else if(s == Status.Failed)
            System.out.println("Try Again");
        else if(s ==Status.Pending)
            System.out.println("Please Wait");
        else
            System.out.println("Done");
    }

}

## package Sky;

enum Status
{
    Running,Failed,Pending,Success;
    }

public class Sky {
    public static void main(String[]args)
    {
        Status s = Status.Running;

        if(s == Status.Running)
            System.out.println("All Good");
        else if(s == Status.Failed)
            System.out.println("Try Again");
        else if(s ==Status.Pending)
            System.out.println("Please Wait");
        else
            System.out.println("Done");
    }

```

}

HTML.
Level 0
1. IDE or Code Editor
1.What is IDE
2. Need of IDE
3. IDE Selection
4. Installation Extension
vs code Extensions

2.Wesbsite Components and Fundamentals.

1.Client Side vs Server Side
2. FrontEnd/BackEnd/FulStack
3. Role of Browser
4. HTML
5. CSS
6. JS

1. IDE or Code Editor.

1.1 What is IDE
1. IDE Stands for Integrated Devlopment Environment.
2. Software suite that consolidates basic tools required for Software development.

3. Central hub for coding, finding problems, and testing.
4. Designed to improve developer efficncy.

1.2 Need of IDE
1. Streamlines development.
2. Increases productivity.
3. Simplifies complex tasks.
4. Offers a unified workspace.
5. IDE Features

1. code Autocomplete
2. Syntax Highlighting
3. Vesion Control
4. Error Checking

1.3 IDE Selection
1. Sublime Text
2. Atom
3. VS Code
4. Github CodeSpaces

1.4 Installation & Setup

1.Search VS Code

1.5 VSCode Extensions
1. Live Server
2. Pretter

Lelvel 0.
2. Website components And Fundamentals

2.1 Client Side vs Server Side.

	Client Side	Server Side
Execution Location	Executes on user's device.	Executes on a remote machine.

Languages	primarily java script,HTML,CSS.	PHP,Python,java Node.js,etc
Main Job	Makes clicks and scrolls work	Manages saved information
Access Level	can't access server data	can read/write file interact with
Speed	Quicker for UI changes	Slower due to network latency

2.2 FrontEnd/BackEnd/FullStack

2.3 Role of Browser

1. Displays web page;- Turns HTML code into what you see on screen.
2. User Clicks:- Helps you interact with the web page.
3. Updates Content:- Allows changes to the page using javaScript.
4. Loads Files:- Gets HTML, images,etc,from the server.

2.4 HTML (Hypertext Markup Language)

1. Structure:- Sets up the layout.
2. Content:- Adds text, images, links.
3. Tags:- Uses elements like <p>,<a>.
4. Hierachy:- Organizes elements in a tree.

2.5 CSS (cascading style sheets)

1. Style:- Sets the look and feel.
2. colors & Fonts:- Customize text and background.
3. Layout:- Controls position and size.
4. Selectors:- Targets specific HTML elements.

2.6 Js (Java Script)

1. javaScript has nothing to do with java.
2. Actions:- Enables interactivity.
3. Updates:- Alters page without reloading.
4. Events:- Responds to user actions.
5. Data:- Fetches and sends info to server.

LEVEL 1
HTML Basics
1. Starting up
1. First File using Text Editor
2. File Extensions
3. Opening the project in VsCode
4. Index.html

2.Basics of HTML
1. What are Tags
2. Using Emmett ! to generate code
3. Basic HTML page
4. MDN Documentation
5. Comments
6. Case Sensitivity

1.1 Firs file using Text Editor

1. Create a folder with name First Project on your Desktop.
2. Open Notepade.
3. Create a file and sace it as index.html

4149 **4. Copy Sample code**
4150 **5. Open Browser and Check.**
4151
4152